# Foundations of Bind-Aware Shrinking: A Categorical and Structural Approach to Test-Case Minimization

## 1. The Architectural Paradigm of Property-Based Testing and the Shrinking Problem

Property-Based Testing (PBT) represents a paradigm shift in software verification, moving from manually constructed, point-based unit tests to the specification of universal invariants that must hold across a vast, automatically generated input space. Traditional PBT frameworks, heavily influenced by Haskell's QuickCheck, rely on type-directed random data generators to supply these inputs.[1] When an input falsifies a property, the framework invokes a "shrinking" phase. Shrinking is the automated mechanism through which the testing framework simplifies the failing test case to isolate the minimal reproducible counterexample, thereby accelerating debugging and fault localization.[3]

In classical PBT architectures, shrinking operates almost exclusively in the output domain. The generator produces a concrete data structure, and upon a property failure, the framework applies heuristic, type-specific mutations to this output—such as removing elements from a list, decrementing integers, or simplifying strings.[4] While this output-domain minimization is highly effective for flat or independent data structures, it catastrophically breaks down in the presence of dependent generation, specifically when generators are composed using monadic bind operations.[5]

### 1.1 The Monadic Bind Bottleneck

The monadic bind operator (>>=) allows the output of an upstream generator to dynamically dictate the structure of a downstream generator.[6] For example, a generator might first produce an integer $N$, and subsequently generate a list of $N$ highly constrained tree structures. In a traditional output-domain shrinker, attempting to shrink the upstream integer $N$ completely invalidates the structural integrity of the downstream list, as the framework lacks the semantic context to relate the mutated integer back to the internal state of the downstream generator. The functional closures that encapsulate these data-flow dependencies are entirely opaque to the framework, rendering the shrinker "bind-unaware."

Because the framework cannot see inside the closures, it is forced to either discard valid shrinking pathways or blindly guess new downstream values, resulting in an exponentially large search space.[7] Consequently, traditional shrinkers waste vast computational resources exploring invalid or suboptimal states, often failing to reach the true minimal counterexample.[8]

### 1.2 Reification, Purity, and the Swift @Sendable Constraint

Solving the bind-aware shrinking problem necessitates a fundamental architectural shift: the testing framework must operate on the input domain (the sequence of random choices, or entropy) rather than the output domain (the final generated value).[9] To achieve this, the generator's structure must be reified into an inspectable Abstract Syntax Tree (AST) that explicitly models both data and control dependencies.

In the context of the Swift programming language, this structural reification is uniquely empowered by the @Sendable closure attribute. The @Sendable constraint enforces strict data-isolation, guaranteeing that closures do not capture mutable reference semantics or external state. When applied to the functional closures within the reified generator AST, @Sendable guarantees that the generation process is functionally pure. This purity ensures that the resulting AST is a strictly acyclic Directed Acyclic Graph (DAG) of data dependencies.

By establishing a pure, reified AST equipped with invertible maps, the shrinking process can be elevated from ad-hoc heuristics to a rigorous mathematical discipline. This report investigates principled analogues from formal computer science—specifically compiler optimization, program slicing, reversible computing, bidirectional transformations, and category theory—to design an exhaustive theoretical framework and a systematic algorithm for bind-aware shrinking. The objective is to construct an algorithm that systematically minimizes upstream structure before attempting to minimize dependent downstream values, guaranteeing absolute minimality and deterministic performance.

## 2. Categorical Semantics of Optimization Reductions

To formalize the structural minimization of a reified AST, the shrinking process must be modeled mathematically. Category theory provides a robust framework for reasoning about compositional structures, specifically through the semantics of optimization reductions.[11]

### 2.1 Optimization Problems as Costed Sets

In the categorical semantics of optimization, an optimization problem $P$ is defined as an object presented as a "costed set".[11] This is formally denoted as $P = (X, c_P)$, where $X$ represents the decision set (the space of all candidate solutions) and $c_P : X \to \overline{\mathbb{R}}$ is an extended-real objective function that assigns a scalar cost to each decision.[11] Infeasible points are strictly assigned a cost of $+\infty$.[11]

In the context of bind-aware shrinking, the optimization object $P$ maps directly to the test-case reduction problem:

- **Decision Set ($X$):** The space of all possible entropy streams (trees of random choices) that drive the reified generator AST.
- **Objective Function ($c_P$):** A complexity metric evaluating the structural weight of the

AST. If the generated value passes the property test, it is considered an invalid shrink target, yielding $c_P(x) = +\infty$. If the property fails, $c_P(x)$ yields a finite real number representing the hierarchical size and scalar magnitude of the generated data.

- **Global Minimizers ($Sol(P)$):** The subset of failing choice trees that possess the absolute minimum structural complexity, formally defined as $\{x \in X : c_P(x) = \inf c_P(x)\}$.[11]

## 2.2 Certified Reductions and Solution Recovery

Morphisms in this category are defined as certified (exact) reductions equipped with a solution recovery mechanism.[11] A reduction morphism $a : P \to Q$ represents an algorithmic transformation that maps instances of problem $P$ into instances of problem $Q$.
A certified deterministic reduction consists of an encoding map ($enc_a : X \to Y$) and a decoding map ($dec_a : Y \to X$).[11] For the reduction to be considered "certified," the costs of these maps must be monotone in the appropriate directions, ensuring that $c_Q(enc_a(x)) \leq c_P(x)$ and $c_P(dec_a(y)) \leq c_Q(y)$.[11] The decoding map serves as the "solution recovery" mechanism, translating the solver's output in the reduced space back into a valid solution for the original problem.[11]
In the PBT shrinker, a reduction morphism corresponds to a proposed structural truncation or value decrement applied to the reified AST. The solution recovery mechanism is the backward propagation of this proposed shrink through the generator's invertible maps, translating the optimized sub-structure back into a globally valid entropy stream.[11] The requirement that solutions map back along recovery maps is formalized as a natural transformation between functors representing the feasible and solution sets.[11]

## 2.3 The Kleisli Generalization for Search Spaces

Because shrinking involves exploring a vast, non-deterministic search space of potential AST truncations, deterministic exact reductions are insufficient. The categorical framework addresses this by generalizing into the Kleisli category of a monad $T$.[11]
To model the non-deterministic exploration of the reified AST, the framework utilizes the powerset monad $\mathcal{P}$. A Kleisli arrow $f : X \to \mathcal{P}(Y)$ maps a single failing choice tree to a multitude of potential reduced trees.[11] The composition of these non-deterministic reductions relies on Kleisli lifting, ensuring that objective-value bounds propagate correctly across multiple shrinking passes.[11]
Furthermore, strict exactness is often too rigid for heuristic search. The framework accommodates this through approximate reductions, where the objective-value guarantees

are relaxed using affine slack parameters.[11] A multiplicative slack $\alpha$ and an additive slack $\beta$ allow the recovery map to satisfy $c_P(dec_a(y)) \leq \alpha c_Q(y) + \beta$.[11] These slack parameters form a grade monoid $G = Aff_{\geq 0} \times W$, where $W$ tracks computational resources (e.g., test execution time). This unified graded structure ensures that the shrinker systematically balances the magnitude of structural complexity reduction against the computational cost of evaluating the property.[11]

| Categorical Concept | Optimization Reduction Framework | Bind-Aware Shrinking Analogue |
|---|---|---|
| **Object ($P$)** | Costed Set $(X, c_P)$ | The space of entropy streams and their structural complexity. |
| **Morphism ($a$)** | Certified exact/approximate reduction | A proposed truncation/decrement on the reified AST. |
| **Decoding Map ($dec_a$)** | Solution recovery mechanism | Backward propagation of the shrink to the entropy stream. |
| **Monad ($T$)** | Kleisli generalization (e.g., Powerset $\mathcal{P}$) | Non-deterministic exploration of candidate shrinks. |
| **Grade Monoid ($G$)** | Tracking slack ($\alpha, \beta$) and resources ($W$) | Balancing structural reduction size vs. test execution time. |

# 3. Bidirectional Transformations and Monadic Profunctors

To realize the solution recovery mechanism mathematically described above, the Swift PBT framework must be capable of executing the generation process in reverse. This necessitates the integration of Bidirectional Transformations (BX) and the specific functional abstraction of monadic profunctors.

## 3.1 Principles of Bidirectional Transformations (BX)

Bidirectional transformations are programs wherein a single piece of code dictates a relationship that can be executed in multiple directions, maintaining consistency between different representations of overlapping information.[12] In computer science, BX is most commonly formalized through the concept of lenses.[12]

A standard asymmetric lens consists of two functions operating between a source state $S$ and a view state $V$ [12]:

1. **get:** A forward transformation mapping $S \to V$.

2. **put:** A backward transformation mapping an updated view and the original source back to an updated source: $V \times S \to S$.

To ensure semantic correctness, a well-behaved lens must satisfy foundational laws [12]:

- **GetPut (Hippocracy):** If no changes are made to the view, the source must remain unchanged: $\texttt{put}(S, \texttt{get}(S)) = S$.
- **PutGet (Correctness):** If a valid view is put back into the source, getting the view from that new source must yield the updated view: $\texttt{get}(\texttt{put}(S, V')) = V'$.
- **PutPut (History Ignorance):** Sequential updates to the view overwrite previous updates: $\texttt{put}(\texttt{put}(S, V'), V'') = \texttt{put}(S, V'')$.

In the context of the PBT shrinker, the get function corresponds to the forward generation of a Swift data structure from an entropy stream. The put function corresponds to the shrinking phase: taking a minimized data structure (the updated view) and propagating those constraints backward to calculate a minimal entropy stream (the updated source).[13] Because monadic generators entail complex structural dependencies where neither state space fully determines the other, the framework utilizes symmetric lenses (or optics), which relate two states through a shared minimal complement.[12]

## 3.2 Monadic Profunctors and the comap Operation

While lenses provide a robust foundation for static data structures, property-based generators utilize dynamic, effectful computations (monads) to introduce randomness. Reconciling the covariant nature of monads with the contravariant requirements of bidirectional lenses is achieved through **monadic profunctors**.[6]

A profunctor $P$ is a binary type constructor that acts as a contravariant functor in its first parameter and a covariant functor in its second.[5] A monadic profunctor integrates this bidirectional capability with monadic sequencing.[2] In the PBT framework, a bidirectional generator (a "biparameterized" structure) unifies the forward and backward computations [6]:

$$\texttt{Fwd}\ m\ u\ v = m\ v$$

$$\texttt{Bwd}\ m\ u\ v = u \to m\ v$$

The forward component (Fwd) utilizes a standard state monad to consume random entropy and produce a value.[6] The backward component (Bwd) utilizes a writer monad to reconstruct the entropy trace from a provided value.[6]

The critical operation that enables the backward flow of data across the profunctor is comap.[2] While a standard covariant fmap applies a function $(a \to b)$ to a functor containing $a$, the

contravariant comap applies a function $(c \to b)$ to a profunctor producing $a$, effectively pre-composing the extraction of a sub-component:

$$\texttt{comap} :: (c \to \text{Maybe } b) \to P\ b\ a \to P\ c\ a$$

During the shrinking phase, comap serves as a focusing mechanism. When a value flows backward through a reified bind node, comap isolates the specific sub-part of the generated structure that the left side of the bind must reflect upon.[2] If this focusing operation succeeds, the left side of the bind recovers its random choices, allowing the bidirectional reflection to continue seamlessly up the AST.[2]

### 3.3 Invertible Bind and Quasicompositionality

The primary theoretical hurdle in monadic bidirectional programming is ensuring that the bind operator (>>=) maintains "round-tripping" correctness. In a standard monad, information can easily be lost across a bind, rendering the backward pass impossible. To guarantee that the generation sequence can be inverted, the shrinker relies on the mathematical property of **injective arrows** and **quasicompositionality**.[6]

A function $k : v \to m\ w$ used within a bind operation is defined as an injective arrow if there exists a deterministic left inverse $k' : w \to v$.[6] This guarantees that the intermediate monadic output $w$ contains sufficient structural information to uniquely recalculate the input $v$:

$$k(x) \gg= \lambda y \to \text{return}(x, y) \equiv k(x) \gg= \lambda y \to \text{return}(k'(y), y)$$

Quasicompositionality dictates that if a bidirectional property (such as correctness) holds for a generator $P$, and the function $k$ is strictly an injective arrow, then the property holds globally for the composed generator $(P \gg= k)$.[6] Because the shrinker operates on a reified AST enforced by Swift's @Sendable purity constraints, all data-flow dependencies are strictly localized and trackable, ensuring that the injective arrow property holds. This allows the shrinker to safely and deterministically invert the bind operation, transforming standard generators into **reflective generators** capable of recovering the exact entropy choices that produced any given value.[2]

## 4. Contextual Propagation via Comonadic Semantics

While monads and monadic profunctors provide the mechanism for sequential generation and its direct inversion, test-case reduction is inherently a holistic process. Modifying a random choice deep within an upstream branch of the AST fundamentally alters the contextual environment of the entire downstream graph. To formalize how local state minimizations are propagated to the global structural context, the algorithm relies on the categorical dual of

monads: **comonads**.[20]

## 4.1 The Dual Structure: Comonads and Cobind

In category theory, a monad encapsulates a computation that produces a value within a specific context (e.g., optionality, state, or lists). Conversely, a comonad encapsulates a value that is strictly situated within a surrounding context or environment.[21] While a monad builds up computations sequentially, a comonad extracts values from a context and extends local transformations across the entire global structure.[20]

A comonad is defined by a parameterized type $\alpha\ \texttt{com}$ equipped with two fundamental operations [20]:

1. **extract (or proj):** $\forall \alpha . \alpha\ \texttt{com} \rightarrow\ \alpha$. This operation retrieves the focused value from its surrounding context.
2. **cobind (or extend):** $\forall \alpha, \beta . (\alpha\ \texttt{com} \rightarrow\ \beta) \rightarrow \alpha\ \texttt{com} \rightarrow \beta\ \texttt{com}$. This operation takes a local transformation that computes a new value based on the current context, and applies that transformation universally across all possible contexts within the structure.

For a structure to be a valid comonad, it must satisfy precise algebraic laws [20]:

- $\texttt{extract}(\texttt{cobind}(k, x)) =\ k(x)$
- $\texttt{cobind}(\texttt{extract}, x) =\ x$
- $\texttt{cobind}(k_2 \circ \texttt{cobind}(k_1)) = \texttt{cobind}(k_2) \circ \texttt{cobind}(k_1)$

## 4.2 Applying cobind to the Reified AST

In the context of the bind-aware shrinker, the reified AST—along with the specific stream of entropy that traverses it—functions as the comonadic environment. Each node within the AST is not an isolated generator; it is a generator firmly embedded within the global execution trace of the property test.[23]

When the shrinker identifies a potential structural minimization (e.g., truncating an array generation sequence), it performs this optimization locally. However, this local change invalidates the absolute positional indexing of the entropy stream for all subsequent operations. The cobind operation solves this by taking the localized structural update (the function $k$) and systematically "extending" it across the entire AST environment.

Through cobind, the shrinker calculates exactly how the localized truncation shifts the contextual boundaries of the remaining entropy.[20] It safely patches the global entropy stream, excising the bytes corresponding to the deleted structural branch while perfectly preserving the alignment and context of the downstream entropy. This guarantees that when the minimized entropy stream is evaluated forward, the remaining downstream values are reconstructed exactly as they were, completely insulating them from the upstream structural shift.

| Operation Category | Primitive | Function Signature | Role in Bind-Aware Shrinking |
|---|---|---|---|
| **Monadic** | bind | $m\ a \to (a \to m\ b)$ | Sequentially composes data-dependent generators. |
| **Bidirectional** | comap | $(c \to \mathrm{Maybe}\ b) \to$ | Focuses on sub-components to invert the bind sequence. |
| **Comonadic** | extract | $\alpha\ \mathtt{com} \to \alpha$ | Isolates a specific AST node from the global trace. |
| **Comonadic** | cobind | $(\alpha\ \mathtt{com} \to \beta) \to \alpha$ | Propagates a local AST truncation to the global entropy state. |

### 4.3 Join Inverse Rig Categories

The integration of monadic generation, bidirectional comap, and comonadic cobind forms a holistic system for reversible functional programming. Recent advancements in categorical semantics have unified these concepts under **join inverse rig categories**.[25] These categories act as the natural mathematical marriage of join inverse categories and rig categories, providing an adequate model for reversible pattern-matching and deterministic backward semantics.[25] By adhering to the axioms of join inverse rig categories, the shrinker's internal representation maintains absolute topological consistency, ensuring that the reduction of the AST is both sound and complete.

## 5. Program Slicing: Isolating Dependency Paths

Before the shrinker can leverage comap and cobind to optimize the AST, it must determine *which* structural nodes actually contributed to the property failure. Minimizing random choices that have no bearing on the test's outcome is mathematically useless and computationally expensive. To isolate the relevant dependency paths, the framework adapts principles from **Program Slicing**.[28]

### 5.1 Static vs. Dynamic Backward Slicing

Program slicing, originally conceived by Mark Weiser, is a technique for computing a subset of program statements that influence a specific point of interest, known as the slicing criterion.[28] The slicing criterion is defined as a tuple $C = (x, v)$, where $x$ is a statement location and $v$ is a variable of interest.[29]

Slicing is bifurcated into two primary paradigms [31]:

- **Static Slicing:** Computes the slice without making assumptions regarding the program's

input.[31] Because it must account for all possible execution paths, static slices are over-approximations that require computationally expensive reachability analysis over abstract dependency graphs.[33]

- **Dynamic Slicing:** Computes the slice for a specific, concrete execution trace.[31] It identifies only the statements that *actually* affected the variable's value during a particular run, resulting in a highly precise, minimal subset of the program.[30]

In the context of PBT shrinking, the slicing criterion $C$ is the boolean assertion that failed during the test execution. Because the shrinker operates on a specific, failing instantiation of the generator, the exact analogue required is **dynamic backward slicing**.[31] The shrinker traverses the execution trace backward from the failing assertion to identify the exact random choices in the AST that precipitated the failure.[34]

## 5.2 Control and Data Dependence Tracking

Dynamic slicing operates by traversing two intertwined types of dependencies within the Dynamic Dependence Graph (DDG) [36]:

1. **Data Dependence:** Occurs when the output of one node is directly consumed by another. In the reified AST, data dependence is explicitly modeled by the parameter passing between bind closures.[37] If node $B$ is generated based on a value produced by node $A$, a data dependence edge connects them.
2. **Control Dependence:** Occurs when the execution of a node is conditionally gated by another node. In the AST, this manifests in branching structures (e.g., if/else paths or the length of a generated array).[36] If the number of child nodes in a list generator is determined by an upstream integer node $N$, the child nodes are control-dependent on $N$.

## 5.3 Executable Slices and Mass Pruning

A fundamental requirement of a program slice is that the resulting subset must constitute an independently executable program that preserves the behavior of the original program with respect to the slicing criterion.[28] For the shrinker, the "executable program" is the modified generator AST, and the "preserved behavior" is the continuous falsification of the property test. By executing a dynamic backward slice on the reified AST, the shrinker partitions the nodes into two sets: those on the slice (relevant to the failure) and those off the slice (irrelevant). Utilizing the principle of executable slices, the algorithm immediately executes a **mass pruning** operation. Every random choice node that does not exist on the dynamic slice is instantly clamped to its theoretical absolute minimum (e.g., integers to zero, strings to empty, enums to their base case).[40]

Because these nodes are mathematically proven to have no control or data dependence on the failing assertion, clamping them cannot possibly resolve the failure. This program slicing pass

allows the shrinker to safely eliminate vast swaths of the entropy search space in $O(1)$ operations per node, achieving an immediate and profound structural minimization before the iterative search phase even begins.[36]

# 6. Compiler Optimization Analogues for Structural Minimization

Once the AST is pruned via dynamic slicing, the shrinker must iteratively search for deeper reductions. This iterative process maps directly to the passes executed by an optimizing compiler.[42] While a compiler transforms an AST to minimize execution time or memory footprint, the shrinker transforms the AST to minimize the topological complexity of the generated data.[42]

## 6.1 Structural Minimization vs. Value Minimization

A cardinal principle in the design of optimizing compilers is the strict segregation of optimization passes into structural modifications and value modifications.[43]

- **Structural Optimization:** Passes that alter the topology of the control-flow or data-flow graphs. Examples include dead code elimination, loop unrolling, and branch folding.[43]
- **Value Optimization:** Passes that simplify scalar computations within a fixed structural topology. Examples include constant folding and strength reduction.[42]

This distinction is the fundamental thesis of a bind-aware shrinker: **structural minimization must systematically precede value minimization.**

Consider a generator AST that produces an array of $N$ elements, where each element is a complex nested struct. The upstream node $N$ dictates the overall topology (the structure). The downstream nodes generate the scalar data (the values) within the structs. If a traditional shrinker attempts to minimize the downstream scalar values first, it wastes thousands of CPU cycles optimizing data that might subsequently be entirely deleted if the upstream node $N$ is reduced.

Furthermore, because of the bind dependency, altering the upstream structure $N$ fundamentally shifts the alignment of the underlying entropy trace.[46] If the algorithm minimizes values first, any subsequent structural shrink will destroy the careful value optimizations. Therefore, by modeling the shrinker after a compiler pipeline, the algorithm enforces a strict topological ordering: it exclusively targets upstream structural nodes, repeatedly applying comap and cobind to truncate branches, until the AST reaches a local topological minimum. Only then does the algorithm transition to value minimization passes on the leaf nodes.[48]

## 6.2 Partial Evaluation as Test-Case Reduction

The most powerful compiler analogue for structural test-case reduction is **Partial Evaluation** (or program specialization).[47] Partial evaluation is an algorithm which, when given a program

and a subset of its known input data, produces a specialized "residual program".[50] This is formally rooted in recursive function theory via Kleene's s-m-n theorem.[50]
In the context of the PBT shrinker, the "program" is the reified generator AST, and the "known inputs" are the fixed entropy choices derived from the dynamic slice. By treating the generator as an interpreter for random data, the shrinker partially evaluates the AST with respect to the specific random choices that govern the failing control flow.[51]
This partial evaluation statically resolves branches of the generator that are unaffected by the current shrinking pass, compiling them down to constant values.[47] The resulting residual AST is a vastly simplified, structurally flat representation of the generator that is explicitly specialized for the specific property failure.[53] This specialized representation drastically reduces the cyclomatic complexity of the subsequent reverse-mode search passes, maximizing computational efficiency.

| **Compiler Optimization** | **Mechanism** | **Analogue in Bind-Aware Shrinking** |
|---|---|---|
| **Dead Code Elimination** | Removing unreached branches. | Pruning AST nodes not on the dynamic backward slice. |
| **Partial Evaluation** | Specializing programs on known inputs. | Compiling non-focal AST branches into fixed constant values. |
| **Loop Unrolling / Reduction** | Altering control-flow topology. | Truncating upstream dependent collection lengths. |
| **Constant Folding** | Minimizing arithmetic expressions. | Minimizing scalar leaf values after structural passes complete. |

# 7. Reversible Computing and Backward State Propagation

When the shrinker successfully finds a simpler structural configuration for an upstream node, it must update the underlying entropy state without corrupting the un-targeted downstream dependencies. This requirement bridges the framework into the domain of **Reversible Computing**.

## 7.1 Thermodynamic Limits and Landauer's Principle

Reversible computing is a paradigm wherein every computational step is strictly time-reversible; given the output of an operation, it is mathematically possible to perfectly reconstruct the input.[54] This field emerged from the thermodynamic limits of conventional computing, specifically **Landauer's Principle**, which states that the erasure of information inevitably leads to an increase in physical entropy (heat dissipation).[55]
To avoid the permanent loss of state associated with non-reversible operations, reversible architectures rely on a **Landauer embedding**.[56] Rather than overwriting variables destructively,

a Landauer embedding maintains a rigorous "history" or "reverse window" of the computational path, preserving the exact information necessary to trace the system backward through time.[56] In the architecture of the bind-aware shrinker, the raw, untyped stream of random bytes (the entropy) functions as the physical substrate, and the reified AST acts as the Landauer embedding. By rigidly mapping every AST node to the precise bytes of entropy it consumed during the forward pass, the system preserves an exact history of the state transitions.[58] The strict immutability enforced by Swift's @Sendable closures ensures that no hidden side-effects can "erase" information outside the bounds of the Landauer embedding.

### 7.2 Reverse-Mode Automatic Differentiation Analogue

The mechanical process of updating the entropy state through the Landauer embedding is strongly analogous to **reverse-mode automatic differentiation (AD)**.[59] In modern deep learning, reverse-mode AD computes gradients by starting at the scalar loss function and propagating the derivatives backward through the computational graph via the chain rule.[60] In the PBT framework, the "loss function" is the objective cost of the optimization problem $c_P(x)$ (the structural size of the failing case). The shrinker computes a "gradient of minimization"—a proposed discrete truncation at a specific AST node. Instead of propagating continuous derivatives, the shrinker propagates discrete structural constraints backward through the DAG.[61]

Utilizing the Bwd component of the monadic profunctor (comap), the proposed reduction is pulled backward from the target AST node down to the root of the entropy stream.[6] As this discrete "gradient" flows backward, the cobind operation recalculates the exact alignment offsets for the entire entropy trace. This reverse-mode propagation ensures that when the upstream structural bytes are modified or deleted, the bytes belonging to the downstream dependencies are shifted into precise alignment, perfectly preserving their localized state.[63] This capability allows the shrinker to perform massive topological surgery on the generator structure without inducing catastrophic forgetting in the child nodes.

## 8. Synthesis: The Principled Bind-Aware Shrinking Algorithm

By synthesizing the categorical semantics of optimization reductions, the bidirectional mapping of monadic profunctors, the contextual propagation of comonads, the dependency isolation of program slicing, and the state-conservation of reversible computing, we arrive at a fully principled algorithm for bind-aware shrinking in Swift.

The algorithm executes in four distinct phases:

### Phase 1: Reification and Landauer Tracing

1. **AST Generation:** The user defines generators using a monadic API. Thanks to Swift's @Sendable attribute, these closures are guaranteed pure. The library reifies these definitions into an explicit AST where every node is a monadic profunctor (implementing

Fwd, Bwd, and comap).
2. **Entropy Embedding:** The generator executes a forward pass, consuming a raw entropy stream to produce a concrete Swift value. The system records a Landauer trace, explicitly mapping each AST node to the exact byte indices it consumed.

### Phase 2: Dynamic Slicing and Mass Pruning

1. **Execution-Aware Slicing:** The generated value is evaluated against the property. Upon failure, the framework computes a dynamic backward slice from the failing assertion up through the AST, identifying the precise data and control dependencies.
2. **Dead Branch Elimination:** Any AST node that is not present on the dynamic slice is instantly clamped to its theoretical structural minimum. This immediately collapses vast regions of the search space without requiring iterative testing.

### Phase 3: Topological Structural Minimization

1. **Topological Sorting:** The AST dependency graph is topologically sorted. The algorithm enforces the compiler optimization rule: upstream structural nodes are strictly prioritized over downstream value nodes.
2. **Iterative Reduction:** The algorithm targets the highest un-minimized structural node. It proposes a discrete reduction (e.g., truncating a collection length).
3. **Reverse-Mode Propagation:** The proposed reduction acts as a loss gradient. Using comap, the constraint is pulled backward. cobind is invoked to shift the global entropy alignment, preserving the specific bytes allocated to the downstream dependent nodes.
4. **Evaluation:** The modified entropy is pushed through a forward pass. If the property still fails, the structural reduction is permanently committed. This process loops until all structural nodes reach a local topological minimum.

### Phase 4: Value Minimization

1. **Scalar Folding:** Once the macroscopic topology of the AST is frozen in its minimal failing state, the algorithm proceeds to the leaf nodes.
2. **Constant Reduction:** Standard value-based heuristics (e.g., halving integers, simplifying characters) are applied to the leaves, utilizing the same reverse-mode propagation to update the underlying entropy bytes.

## 9. Conclusion

The problem of shrinking dependent, monadic data structures in Property-Based Testing has long been constrained by the limitations of output-domain heuristics. By treating functional closures as opaque black boxes, legacy frameworks are fundamentally incapable of maintaining state alignment when upstream structural dependencies are modified.
The framework delineated in this report overcomes these limitations by elevating the shrinking process to a problem of categorical and structural optimization. By reifying the generator into a pure AST—secured by Swift's @Sendable concurrency constraints—the system gains the ability to inspect and manipulate the exact mathematical dependencies that govern generation.

Integrating monadic profunctors and bidirectional lenses (comap) allows the generator to run backward, while comonadic operations (cobind) ensure that local structural minimizations are safely propagated to the global entropy context.
Furthermore, by mapping the reduction process to principles of compiler optimization and program slicing, the algorithm establishes a rigorous, mathematically sound order of operations: isolating dependencies via dynamic backward slicing, and systematically enforcing upstream structural minimization before downstream value minimization. Supported by the state-conservation principles of reversible computing, this architecture guarantees that catastrophic state invalidation is entirely eliminated. The resulting bind-aware shrinker operates with unparalleled deterministic precision, systematically reducing the most complex, dependent Swift data structures to their absolute theoretical minimal forms.

**Referanser**


1. Property-Based Testing –– Call for Opinions, Comments - Development - scsynth, brukt mars 18, 2026, https://scsynth.org/t/property-based-testing-call-for-opinions-comments/9087
2. Reflecting on Random Generation - Harrison Goldstein, brukt mars 18, 2026, https://harrisongoldste.in/papers/icfp23-reflective.pdf
3. Shrinking - Kotest, brukt mars 18, 2026, https://kotest.io/docs/proptest/property-test-shrinking.html
4. Your own property based testing framework — Part 3: Shrinkers | by Nicolas Dubien, brukt mars 18, 2026, https://medium.com/@nicolasdubien/your-own-property-based-testing-framework-part-3-shrinkers-564fa7a180eb
5. Composing Bidirectional Programs Monadically.pdf - Kent Academic Repository, brukt mars 18, 2026, https://kar.kent.ac.uk/72625/11/Composing%20Bidirectional%20Programs%20Monadically.pdf
6. Composing bidirectional programs monadically - Meng Wang, brukt mars 18, 2026, https://mengwangoxf.github.io/Papers/ESOP19.pdf
7. Using metaprogramming techniques to enhance eclingo performance through the reification format - DigitalCommons@UNO, brukt mars 18, 2026, https://digitalcommons.unomaha.edu/cgi/viewcontent.cgi?article=1003&context=compscistudent
8. Test-Case Reduction via Test-Case Generation: Insights from the, brukt mars 18, 2026, https://discovery.researcher.life/article/test-case-reduction-via-test-case-generation-insights-from-the-hypothesis-reducer-tool-insights-paper/5a311f77b86a3d10bef70b08910713ef
9. Test-Case Reduction via Test-Case Generation: Insights from the Hypothesis Reducer - Imperial College London, brukt mars 18, 2026, http://www.doc.ic.ac.uk/~afd/papers/2020/ECOOP_Hypothesis.pdf
10. Notes on Test-Case Reduction | David R. MacIver, brukt mars 18, 2026, https://www.drmaciver.com/2019/01/notes-on-test-case-reduction/

11. categories_of_optimization_reductions.pdf
12. Introduction to Bidirectional Transformations - Computer Science, brukt mars 18, 2026, https://www.cs.ox.ac.uk/people/jeremy.gibbons/publications/ssbx-intro.pdf
13. Bidirectional transformation - Wikipedia, brukt mars 18, 2026, https://en.wikipedia.org/wiki/Bidirectional_transformation
14. The Weird World of Bi-Directional Programming - Computer and Information Science, brukt mars 18, 2026, https://www.cis.upenn.edu/~bcpierce/papers/lenses-etapsslides.pdf
15. Dissertation - Computer Science, brukt mars 18, 2026, https://www.cs.cornell.edu/~jnfoster/papers/jnfoster-dissertation.pdf
16. arXiv:1902.06950v1 [cs.PL] 19 Feb 2019, brukt mars 18, 2026, https://arxiv.org/pdf/1902.06950
17. Ungenerators - Harrison Goldstein, brukt mars 18, 2026, https://harrisongoldste.in/papers/icfpsrc21.pdf
18. PROPERTY-BASED TESTING FOR THE PEOPLE Harrison Goldstein A DISSERTATION in Computer and Information Science Presented to the Fac, brukt mars 18, 2026, https://harrisongoldste.in/papers/dissertation.pdf
19. Property-Based Testing in Practice | Request PDF - ResearchGate, brukt mars 18, 2026, https://www.researchgate.net/publication/379799872_Property-Based_Testing_in_Practice
20. Monads in programming language theory - Xavier Leroy, brukt mars 18, 2026, https://xavierleroy.org/mpri/2-4/monads.2up.pdf
21. (PDF) Categorical semantics for arrows - ResearchGate, brukt mars 18, 2026, https://www.researchgate.net/publication/220676552_Categorical_semantics_for_arrows
22. Categorical semantics for arrows - Group MMM, brukt mars 18, 2026, https://group-mmm.org/~ichiro/papers/arrows.pdf
23. Strict and Lazy Semantics for Effects - Computer Science, brukt mars 18, 2026, https://www.cs.cornell.edu/~akhirsch/publications/strict_and_lazy_semantics_for_effects_tr.pdf
24. SEMANTICS FOR SECURE SOFTWARE - Ross Tate, brukt mars 18, 2026, https://rosstate.org/theses/hirsch-thesis.pdf
25. (PDF) Join Inverse Rig Categories for Reversible Functional Programming, and Beyond, brukt mars 18, 2026, https://www.researchgate.net/publication/357378740_Join_Inverse_Rig_Categories_for_Reversible_Functional_Programming_and_Beyond
26. Join inverse rig categories for reversible functional programming, and beyond - SDU, brukt mars 18, 2026, https://portal.findresearcher.sdu.dk/files/225135161/2105.09929v2.pdf
27. [PDF] Reversible Monadic Computing | Semantic Scholar, brukt mars 18, 2026, https://www.semanticscholar.org/paper/Reversible-Monadic-Computing-Heunen-Karvonen/a10ac84ccbd2158f6f4765cd79ef87a2c267f2c4
28. Program Slicing: A Brief Retrospective - IEEE Computer Society, brukt mars 18, 2026, https://www.computer.org/csdl/journal/ts/2025/03/10870474/24488vCjgIM

29. Program slicing - Wikipedia, brukt mars 18, 2026, https://en.wikipedia.org/wiki/Program_slicing
30. Dynamic Program Slicing - Purdue University, brukt mars 18, 2026, https://www.cs.purdue.edu/homes/xyzhang/fall07/Papers/p246-agrawal.pdf
31. A Survey of Program Slicing Techniques. - SciSpace, brukt mars 18, 2026, https://scispace.com/pdf/a-survey-of-program-slicing-techniques-1u546ok1jr.pdf
32. REPORTRAPPORT, brukt mars 18, 2026, https://faculty.cc.gatech.edu/~harrold/6340/cs6340_fall2009/Readings/tip94.pdf
33. SliceMate: Accurate and Scalable Static Program Slicing via LLM-Powered Agents - arXiv, brukt mars 18, 2026, https://arxiv.org/html/2507.18957v1
34. Dynamic Program Slicing - Emergent Mind, brukt mars 18, 2026, https://www.emergentmind.com/topics/dynamic-program-slicing
35. Dynamic Program Slicing - Columbia University, brukt mars 18, 2026, https://www.cs.columbia.edu/~junfeng/08fa-e6998/sched/readings/slicing.pdf
36. Program Slicing, brukt mars 18, 2026, https://people.eecs.ku.edu/~saiedian/Teaching/814/Readings/intro-program-slicing.pdf
37. Program Analysis Program Slicing - Software Lab, brukt mars 18, 2026, https://www.software-lab.org/teaching/winter2024/pa/lecture_slicing.pdf
38. Lesson 4: Data Flow · sampsyo cs6120 · Discussion #452 - GitHub, brukt mars 18, 2026, https://github.com/sampsyo/cs6120/discussions/452
39. Program Slicing - Software Engineering 421, brukt mars 18, 2026, https://se421-fall2018.github.io/resources/readings/WeiserStaticSlicing.pdf
40. Using the Unravel Program Slicing Tool to Evaluate High Integrity Software - National Institute of Standards and Technology, brukt mars 18, 2026, https://www.nist.gov/document/sqwpdf
41. Weight prioritized slicing based on constraint logic programming for fault localization - PMC, brukt mars 18, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7147748/
42. Optimizing compiler - Wikipedia, brukt mars 18, 2026, https://en.wikipedia.org/wiki/Optimizing_compiler
43. Code Optimization Techniques in Compiler Design | by Purva Wankhade | Medium, brukt mars 18, 2026, https://medium.com/@purva.wankhade21/code-optimization-techniques-in-compiler-design-f8199b212bf4
44. Compiler-Based Code-Improvement Techniques - Department of Computer Science, brukt mars 18, 2026, https://www.cs.tufts.edu/~nr/cs257/archive/keith-cooper/survey.pdf
45. Flattening ASTs (and Other Compiler Data Structures) - Cornell: Computer Science, brukt mars 18, 2026, https://www.cs.cornell.edu/~asampson/blog/flattening.html
46. Test-Case Reduction for C Compiler Bugs - Virtual Server List, brukt mars 18, 2026, https://users.cs.utah.edu/~regehr/papers/pldi12-preprint.pdf
47. The Art and Science of Testcase Reduction for Compiler Bugs - Embedded in Academia, brukt mars 18, 2026, https://blog.regehr.org/archives/639

48. VIA PROGRAM SLICING, brukt mars 18, 2026, https://minds.wisconsin.edu/bitstream/1793/60018/1/TR1296.pdf
49. An Introduction to Partial Evaluation - The University of Texas at Dallas, brukt mars 18, 2026, https://www.utdallas.edu/~gupta/courses/apl/neil.pdf
50. Partial Evaluation - ITU, brukt mars 18, 2026, https://www.itu.dk/~sestoft/pebook/jonesgomardsestoft-a4.pdf
51. Mix: A self-applicable partial evaluator for experiments in compiler generation - Department of Computer Science, brukt mars 18, 2026, https://www.cs.tufts.edu/comp/150FP/archive/neil-jones/mix-partial-evaluator.pdf
52. Type-Directed Partial Evaluation - Tidsskrift.dk, brukt mars 18, 2026, https://tidsskrift.dk/daimipb/article/download/7022/5983/22798
53. PARAMETRIC PROGRAM SLICING - Frank Tip, brukt mars 18, 2026, https://www.franktip.org/pubs/popl1995.pdf
54. Reversible computing - Wikipedia, brukt mars 18, 2026, https://en.wikipedia.org/wiki/Reversible_computing
55. Quantum Foundations of Classical Reversible Computing - MDPI, brukt mars 18, 2026, https://www.mdpi.com/1099-4300/23/6/701
56. Reversible Computations in Logic Programming - PMC - NIH, brukt mars 18, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7342172/
57. A new approach to reversible computing with applications to speculative parallel simulation - I.R.I.S., brukt mars 18, 2026, https://iris.uniroma1.it/retrieve/e3835323-ade5-15e8-e053-a505fe0a3de9/Tesi_dottorato_Cingolani.pdf
58. Reversibility for Efficient Computing - DSpace@MIT, brukt mars 18, 2026, https://dspace.mit.edu/bitstream/handle/1721.1/9464/43483299-MIT.pdf?sequence=2&isAllowed=y
59. Foundation Models for Discovery and Exploration in Chemical Space - arXiv, brukt mars 18, 2026, https://arxiv.org/html/2510.18900v1
60. Demystifying Differentiable Programming: Shift/Reset the Penultimate Backpropagator - CS@Purdue, brukt mars 18, 2026, https://www.cs.purdue.edu/homes/rompf/papers/wang-arxiv18b.pdf
61. Methods for Theoretical Treatment of Local Fields in Proteins and Enzymes | Chemical Reviews - ACS Publications, brukt mars 18, 2026, https://pubs.acs.org/doi/10.1021/acs.chemrev.4c00471
62. The Reversible Residual Network: Backpropagation Without Storing Activations - NIPS, brukt mars 18, 2026, http://papers.neurips.cc/paper/6816-the-reversible-residual-network-backpropagation-without-storing-activations.pdf
63. The Reversible Computing Scaling Path: Challenges and Opportunities - Sandia National Laboratories, brukt mars 18, 2026, https://www.sandia.gov/app/uploads/sites/210/2022/06/ECI22-talk-v7.pdf